
Guia de Estudos Java - Fases 0 a 5

O que foi construido, como cada parte funciona e como explicar a evolucao do projeto.

Consolidado a partir dos seis PDFs de fase, do codigo-fonte, dos READMEs, do pom.xml e das configuracoes de execucao do IntelliJ.

Do primeiro projeto no IntelliJ a API Spring Boot, com orientacao a objetos, Maven, arquitetura em camadas, JPA, H2 e Lombok.

Como usar este guia

A trilha conta uma história: primeiro o programa é criado e executado; depois o código ganha conceitos de orientação a objetos; em seguida, o projeto passa a gerenciar bibliotecas, separa responsabilidades e termina com uma API e redução de código repetitivo.

Para estudar, leia uma fase por vez e abra a classe indicada no IntelliJ. Para apresentar, use os resumos de fala e demonstre os programas na ordem sugerida. O código é a evidência do que foi implementado; os PDFs explicam conceitos e exercícios que nem sempre foram todos codificados.

Fase | Questão respondida | Resultado no projeto

Fase: 0 - IntelliJ | Questão respondida: Como criar, organizar e executar um programa Java? | Resultado no projeto: Projeto Maven com Java 21 e demonstração de polimorfismo com animais.

Fase: 1 - Orientação a objetos | Questão respondida: Como representar comportamentos e responsabilidades diferentes? | Resultado no projeto: Herança, classes abstratas, interfaces, sobrecarga e sobrescrição.

Fase: 2 - Maven | Questão respondida: Como obter bibliotecas e padronizar o build? | Resultado no projeto: Dependência Commons Lang declarada no pom.xml e usada no código.

Fase: 3 - MVC em Java puro | Questão respondida: Como separar entrada, regra de negócio e dados? | Resultado no projeto: Model, Controller, Service e Repository com armazenamento em memória.

Fase: 4 - Spring Boot | Questão respondida: Como transformar a arquitetura em uma API HTTP gerenciada? | Resultado no projeto: POST e GET de clientes, Spring Data JPA e H2 em memória.

Fase: 5 - Lombok | Questão respondida: Como reduzir código repetitivo mantendo clareza? | Resultado no projeto: Getters, setters, construtores, Builder e entidades Cliente/Pedido.

PONTO-CHAVE

Ideia central: cada fase reutiliza o aprendizado anterior. MVC ensina a separação; Spring automatiza a montagem das camadas; Lombok reduz o boilerplate que já foi escrito manualmente.

Vocabulário de leitura

Classe é a definição; objeto é uma instância em memória. Dependência é algo que uma classe precisa para cumprir seu papel. Build é o processo de compilar, testar e empacotar o projeto. Endpoint é um caminho HTTP atendido pela API.

Fase 0 - Criar e executar o projeto

Objetivo: preparar o ambiente e comprovar que o ciclo básico funciona: criar uma classe, executar o método main e observar a saída.

O que o PDF ensina

O material parte do IntelliJ IDEA Community, Java 21 e Maven. Ao criar um projeto, selecionam-se Java, Maven e o JDK 21. O IntelliJ gera a estrutura inicial e sincroniza o Maven, o que pode demorar na primeira execução porque as dependências são baixadas.

Parte | Função

Parte: src/main/java | Função: Código Java da aplicação.

Parte: src/main/resources | Função: Configurações e arquivos que não são código Java.

Parte: src/test/java | Função: Local previsto para testes automatizados.

Parte: pom.xml | Função: Configuração do projeto e das dependências Maven.

Parte: target | Função: Saída gerada pelo build; contém classes compiladas e pacotes.

A primeira classe tem o método public static void main(String[] args), que é o ponto de entrada do programa. Uma instrução como System.out.println escreve uma linha no painel Run. A mensagem "Process finished with exit code 0" indica que o processo terminou sem erro.

O que existe neste repositório

Além da estrutura inicial, a fase já contém um exemplo executável de comportamento polimórfico:

```
Animal animal = new Animal();
Animal cachorro = new Cachorro();
Animal gato = new Gato();

animal.emitirSom();
cachorro.emitirSom();
gato.emitirSom();
```

Animal implementa o som genérico. Cachorro e Gato herdam de Animal e redefinem emitirSom(). Por isso as três variáveis podem ser tratadas como Animal, mas cada objeto imprime a resposta correspondente ao seu tipo real: som genérico, latido e miado.

O código está em src/main/java/Fase0. A configuração chamada "Main" no IntelliJ executa Fase0.Main. Há também org.example.Main, que é o exemplo inicial gerado pela IDE; ele não é o ponto de entrada configurado para empacotar o Spring Boot.

Como explicar esta fase

FALA SUGERIDA

"Na fase inicial configurei um projeto Maven com Java 21 no IntelliJ, entendi onde ficam código, recursos e arquivos gerados e executei uma classe pelo método main. O exemplo de animais já mostra que uma variável pode ter o tipo geral Animal enquanto o objeto concreto escolhe o comportamento executado."

O que demonstrar

1. Abrir Fase0.Main.
2. Apontar a classe-base e as duas subclasses.
3. Executar a configuração "Main" e relacionar cada linha impressa com a implementação chamada.

Fase 1 - Orientação a objetos

Objetivo: modelar variações sem duplicar comportamento e entender quando usar herança, abstração, interfaces, sobrecarga e sobrescrição.

Como o código foi organizado

Os exercícios foram separados em pacotes dentro de Fase1. Classes com nomes iguais, como Animal e Funcionario, podem coexistir porque pertencem a pacotes distintos. Cada pacote destaca um tópico, enquanto Fase1.Main integra vários deles.

Pacote | Exemplos e foco

Pacote: Heranca | Exemplos e foco: Animal, Cachorro, Gato; funcionários com salários fixos; polimorfismo.

Pacote: Abstracao | Exemplos e foco: Funcionario abstrato, gerente/vendedor, Forma, círculo e retângulo.

Pacote: Interface | Exemplos e foco: Contratos Descontavel, Taxavel, Auditavel e a classe Produto.

Pacote: Sobrecarga | Exemplos e foco: Versões de somar e validar com parâmetros diferentes.

Pacote: Sobrescricao | Exemplos e foco: Programa que percorre funcionários abstratos e chama os métodos redefinidos.

Herança e polimorfismo

extends indica que a subclasse reaproveita a estrutura ou o comportamento da superclasse. No exemplo de animais, a superclasse fornece a operação comum; cada subclasse troca o resultado. Polimorfismo é a escolha da implementação de acordo com o objeto real durante a execução.

O PDF também propõe a hierarquia Funcionario -> Gerente/Vendedor. O primeiro exemplo usa salários fixos; mais adiante o código refina o modelo com salário-base, vendas e bônus. A anotação @Override torna explícita a intenção de redefinir um método e permite ao compilador apontar uma assinatura escrita incorretamente.

Abstração: um contrato incompleto com estado comum

Forma é abstrata e declara calcularArea() sem fornecer uma fórmula. Circulo calcula $\pi \times \text{raio}^2$; Retangulo calcula largura x altura. O método concreto imprimirArea() chama o método abstrato, então funciona para qualquer forma concreta. Não se instancia Forma diretamente.

Funcionario também é abstrata. Guarda o nome, valida que não esteja vazio e exige que as subclasses implementem calcularSalario(). Isso compartilha estado e uma regra geral, deixando a fórmula específica para cada funcionário.

Interface: capacidade que várias classes podem assumir

Uma interface declara um contrato sem impor uma hierarquia de classes. Produto implementa três capacidades: dar desconto, calcular imposto e gerar log de auditoria. Como Auditavel fornece gerarLogAuditoria() com default, Produto herda essa implementação sem precisar reescrevê-la.

Na implementação, o preço inicial precisa ser não negativo e o percentual de desconto deve estar entre 0 e 100. O desconto altera o preço armazenado. Depois, o imposto é 12% do preço já descontado. Exemplo: R\$ 100, desconto de 10% -> preço R\$ 90; imposto -> R\$ 10,80. O imposto não é calculado sobre os R\$ 100 originais.

Bonificavel exige calcularBonus() e fornece um método padrão calcularBonusAnual(), que multiplica o bônus mensal por 12. Uma classe pode implementar mais de uma interface; isso permite combinar capacidades sem herança múltipla de classes.

Sobrecarga e sobrescrição não são a mesma coisa

Conceito | O que muda | Exemplo do projeto

Conceito: Sobrecarga (overload) | O que muda: Mesmo nome; parâmetros diferentes. O compilador escolhe pela quantidade ou pelos tipos recebidos. | Exemplo do projeto: somar(int,int), somar(double,double), somar(int,int,int); versões de validar e exibirResumo.

Conceito: Sobrescrição (override) | O que muda: Subclasse fornece implementação própria com a mesma assinatura da superclasse. | Exemplo do projeto: Cachorro.emitirSom(), Gerente.calcularSalario() e Vendedor.calcularBonus().

O tipo de retorno sozinho não cria uma sobrecarga. Dois métodos com mesmo nome e mesmos parâmetros são a mesma assinatura para essa regra; mudar apenas int para double no retorno não resolve.

O exemplo integrado de funcionários

Fase1.Main cria uma lista declarada como List<Funcionario>, com uma instância de Gerente e outra de Vendedor. Os valores são:

- Ana: salário-base R\$ 8.000; bônus mensal de 10% = R\$ 800; bônus anual = R\$ 9.600.
- Bruno: salário-base R\$ 2.500 + 5% de R\$ 20.000 em vendas = R\$ 3.500; bônus mensal de 2% das vendas = R\$ 400; bônus anual = R\$ 4.800.

O método exibirResumo(Funcionario) mostra nome e salário. A versão sobrecarregada com um boolean chama a primeira e, quando solicitado, verifica se o funcionário implementa Bonificavel para mostrar o bônus anual. Assim o exemplo junta abstração, herança, interface, overload, override e polimorfismo.

Como explicar esta fase

FALA SUGERIDA

"A classe abstrata define o que todos os funcionários compartilham e exige que cada tipo calcule seu próprio salário. As interfaces descrevem capacidades como bonificação, desconto e auditoria. Sobrecarga escolhe uma operação pelos parâmetros; sobrescrição troca uma implementação herdada. A lista usa o tipo geral Funcionario, mas executa as regras de Gerente e Vendedor."

Fase 2 - Maven e dependências

Objetivo: deixar que uma ferramenta resolva bibliotecas, versões, compilação e empacotamento de maneira repetível.

O problema que o Maven resolve

Sem Maven, cada pessoa poderia baixar arquivos JAR manualmente, adicioná-los ao classpath e repetir a configuração em cada máquina. No Maven, o projeto declara a biblioteca e sua versão no pom.xml. O Maven baixa também as dependências transitivas e guarda os arquivos no repositório local .m2/repository.

A primeira compilação pode ser mais lenta porque precisa baixar os artefatos. Nas próximas, o cache local evita downloads repetidos. .m2 contém dependências do ambiente do desenvolvedor; não é o local do código-fonte do projeto.

Como ler as coordenadas do projeto

Campo Valor do projeto Significado
Campo: groupId Valor do projeto: org.example Significado: Identificador do grupo que publica o artefato.
Campo: artifactId Valor do projeto: estudos-java Significado: Nome do artefato.
Campo: version Valor do projeto: 1.0-SNAPSHOT Significado: Versão ainda em desenvolvimento.
Campo: Java Valor do projeto: 21 Significado: Versão configurada para compilação e destino.

Para uma dependência, groupId + artifactId + version identificam exatamente qual biblioteca será usada. No exercício, foi adicionada org.apache.commons:commons-lang3:3.14.0. O pacote usa StringUtils.isBlank para demonstrar que não basta o JAR estar baixado: a classe também precisa estar disponível no classpath. Texto vazio ou só com espaços é considerado em branco; "texto" não é.

Dependências e escopos

Uma dependência transitiva é uma biblioteca necessária a outra que o Maven resolve automaticamente. Por exemplo, Spring Web traz componentes como servidor embutido e serialização JSON, embora não sejam declarados um a um.

- compile é o padrão e deixa a biblioteca disponível ao compilar e executar.
- test limita a biblioteca ao ciclo de testes.
- provided deixa a dependência disponível para compilação quando o ambiente final deve fornecê-la. Neste projeto Lombok usa provided.
- runtime é usado pelo H2: a aplicação precisa dele ao executar, mas não para compilar as classes que usam apenas as APIs de JPA.

Comandos e ciclo de build

Comando Efeito

Comando: mvn clean | Efeito: Remove artefatos antigos de target.

Comando: mvn compile | Efeito: Compila o código-fonte.

Comando: mvn test | Efeito: Executa testes automatizados configurados.

Comando: mvn package | Efeito: Gera o pacote JAR/WAR em target.

Comando: mvn install | Efeito: Além de empacotar, instala o artefato no repositório Maven local.

Comando: mvn clean package | Efeito: Limpa e reconstrói o pacote desde o início.

O pom.xml deste projeto

Além do Commons Lang, o arquivo atual declara Spring Boot Web 3.2.0, Spring Data JPA 3.2.0, H2 2.2.224 e Lombok 1.18.30. Assim, o POM documenta a evolução das fases no mesmo projeto. O plugin Spring Boot está configurado para usar Fase4.SpringBoot.AplicacaoSpringBoot como classe principal do JAR empacotado.

O README da fase registra aproximadamente 17,830 segundos em uma execução de mvn clean package. Esse número descreve aquela máquina e aquela execução; não é uma duração garantida.

FALA SUGERIDA

"O Maven centraliza dependências e build no pom.xml. O projeto declara o Java 21 e a biblioteca Commons Lang 3.14.0, que é provada pelo uso de StringUtils.isBlank. O repositório .m2 é o cache local; target recebe as classes compiladas e o JAR."

Fase 3 - MVC em Java puro

Objetivo: entender a separação de responsabilidades antes de deixar um framework criar e conectar as classes.

O que significa MVC neste projeto

MVC vem de Model, View e Controller. Em uma API sem tela, a organização prática se concentra em Controller, Service e Repository. O Model continua representando os dados. Cada camada muda por um motivo diferente:

Entrada -> Controller -> Service -> Repository -> fonte de dados

Camada | Responsabilidade | Implementação

Camada: Model | Responsabilidade: Representar os dados do domínio. | Implementação: Cliente: id, nome e idade.

Camada: Controller | Responsabilidade: Receber a operação e delegar; depois devolver o resultado. | Implementação: ClienteController: cadastrar, listar e buscar.

Camada: Service | Responsabilidade: Concentrar regras e decisões de negócio. | Implementação: ClienteService: valida cliente, nome e idade mínima.

Camada: Repository | Responsabilidade: Guardar, listar e localizar dados. | Implementação: ClienteRepository: mapa ordenado em memória.

Como um cadastro percorre as camadas

1. O Controller recebe nome e idade e cria um objeto Cliente.
2. Ele chama `service.cadastrar(cliente)`.
3. O Service rejeita objeto nulo, nome nulo/vazio ou idade menor que 18, lançando `IllegalArgumentException`.
4. Se os dados passam, o Repository gera um id sequencial e guarda o cliente.
5. O resultado retorna pelas camadas até o chamador.

O Repository usa `LinkedHashMap<Long, Cliente>`, então preserva a ordem de inserção. O id é contado a partir de 1. A busca usa `Optional` no Repository; o Service converte ausência em `null`. `listarTodos()` devolve uma nova lista com os valores armazenados, não o próprio mapa.

Por que a validação fica no Service

"Cliente precisa ter pelo menos 18 anos" é regra de negócio. Se ela ficasse no Controller, outro fluxo que não fosse HTTP teria de copiá-la ou poderia ignorá-la. Colocando-a no Service, qualquer entrada - Controller, tarefa programada ou teste - usa a mesma regra.

O projeto demonstra isso com `CadastroProgramado`, um segundo ponto de entrada que recebe o mesmo `ClienteService` e cadastra sem passar pelo Controller. O Repository permanece independente de como o cadastro começou.

O que MainMVC demonstra

MainMVC constrói as dependências manualmente: Repository, depois Service, depois Controller. Cadastra Ana, tenta cadastrar Bruno com 16 anos e captura a exceção, lista todos, busca o id 1 e demonstra o cadastro de Carla pelo fluxo programado. Ainda não há servidor web, JSON ou banco de dados; o mapa existe somente na memória do processo.

Como explicar esta fase

FALA SUGERIDA

"Na fase 3 eu separei dados, entrada, regras e armazenamento. O Controller encaminha; o Service valida e decide; o Repository persiste. Para praticar a arquitetura sem esconder os passos, montei as dependências com new. O armazenamento ainda é um mapa em memória, e um segundo fluxo comprova que a regra pode ser reutilizada fora do Controller."

Fase 4 - API com Spring Boot

Objetivo: preservar as camadas da fase 3 e acrescentar inicialização automática, HTTP, JSON e persistência JPA.

Como o Spring monta a aplicação

`AplicacaoSpringBoot` chama `SpringApplication.run` e está marcada com `@SpringBootApplication`. Essa anotação combina configuração, auto-configuração e varredura de componentes. Como a classe principal está no pacote `Fase4.SpringBoot`, o Spring encontra os componentes nos subpacotes `Controller`, `Service`, `Repository` e `Model`.

As classes anotadas são registradas como beans. Em vez de o main instanciar tudo manualmente, o Spring cria `ClienteService` e injeta `ClienteRepository`; depois injeta o Service no Controller. O construtor único torna essa dependência explícita e ajuda testes futuros.

Anotação | Função | Está no projeto?

Anotação: `@SpringBootApplication` | Função: Inicializa e configura a aplicação; habilita busca dos componentes. | Está no projeto?: Sim, na aplicação principal.

Anotação: `@RestController` | Função: Expõe métodos HTTP e serializa retornos como dados, normalmente JSON. | Está no projeto?: Sim, no Controller.

Anotação: `@Service` | Função: Identifica a camada de negócio. | Está no projeto?: Sim.

Anotação: `@Repository` | Função: Marca acesso a dados; Spring Data também cria o bean da interface JPA. | Está no projeto?: A interface estende `JpaRepository`, não precisa de implementação manual.

Anotação: `@RequestMapping` | Função: Define o prefixo de rota. | Está no projeto?: Sim: `/clientes`.

Anotação: `@PostMapping` / `@GetMapping` | Função: Ligam métodos aos verbos de criação e consulta. | Está no projeto?: Sim.

Anotação: `@RequestBody` | Função: Converte o JSON recebido em objeto Java. | Está no projeto?: Sim, no cadastro.

Anotação: `@Entity`, `@Id`, `@GeneratedValue` | Função: Mapeiam classe, chave primária e geração de id no JPA. | Está no projeto?: Sim, em Cliente.

Rotas implementadas e ciclo da requisição

POST `/clientes` recebe um corpo JSON, converte-o em Cliente, passa pelo Service e salva no Repository. `@ResponseStatus(HttpStatus.CREATED)` define status HTTP 201 se o método concluir com sucesso. GET `/clientes` chama o Service e retorna a lista, que o Spring serializa como JSON.

```
POST http://localhost:8080/clientes
Content-Type: application/json
```

```
{"nome": "Ana", "idade": 25}
```

```
GET http://localhost:8080/clientes
```

O PDF ensina também `@PathVariable` para dados no caminho, como `/clientes/5`, e `@RequestParam` para query string, como `/clientes?nome=Ana`. Esses conceitos são exemplos de extensão: não há métodos que os usem na API atual.

O material apresenta ainda `@PutMapping` e `@DeleteMapping`, mas a implementação efetiva tem apenas POST e GET. Não há atualização ou remoção implementadas.

JPA, Repository e H2

Cliente é uma entidade JPA. O campo id é a chave primária e o banco gera seu valor. O construtor sem argumentos protegido atende ao mecanismo de persistência; o construtor com nome e idade facilita a criação pela aplicação.

`ClienteRepository` estende `JpaRepository<Cliente, Long>`. O Spring Data fornece operações como `save` e `findAll` sem uma classe concreta escrita à mão. O Service preserva a regra de maioridade antes de salvar.

O arquivo `application.properties` define H2 em memória, porta 8080, usuário `sa`, `ddl-auto=create-drop`, exibição de SQL e console H2 habilitado. Como o banco é em memória, os

dados desaparecem quando a aplicação termina. A opção create-drop cria o esquema para a execução e o remove no encerramento.

ATENCAO

Limite atual: a regra inválida lança `IllegalArgumentException`, mas o projeto não define um tratamento global que a converta em resposta HTTP 400. Se essa exceção escapar, o cliente pode receber erro 500. Para uma API mais completa, seria necessário mapear erros com um handler ou usar validação de entrada com respostas adequadas.

Como explicar esta fase

FALA SUGERIDA

"A fase 4 mantém `Controller`, `Service` e `Repository`, mas deixa o Spring gerenciar a criação e a injeção dos objetos. O `Controller` publica `POST` e `GET` em `/clientes`. JPA mapeia `Cliente` para uma tabela, `JpaRepository` fornece operações comuns e o H2 em memória permite testar sem configurar um servidor de banco externo."

Fase 5 - Lombok e entidades relacionadas

Objetivo: reduzir métodos repetitivos sem perder o entendimento do que eles fazem e demonstrar construção de objetos com vários campos.

O que o Lombok faz

Lombok processa anotações durante a compilação e gera código como getters, setters e construtores. Esses métodos existem no bytecode compilado; não são criados por uma lógica mágica em tempo de execução. A IDE pode precisar do plugin Lombok para reconhecer os métodos gerados durante a edição.

Anotação | Código gerado ou efeito | Uso mostrado no material

Anotação: `@Getter` / `@Setter` | Código gerado ou efeito: Métodos de leitura e alteração de campos. | Uso mostrado no material: Entidades e classes simples.

Anotação: `@NoArgsConstructor` | Código gerado ou efeito: Construtor sem argumentos. | Uso mostrado no material: Entidade `Cliente` e `Pedido`.

Anotação: `@AllArgsConstructor` | Código gerado ou efeito: Construtor com todos os campos. | Uso mostrado no material: `Pedido`.

Anotação: `@RequiredArgsConstructor` | Código gerado ou efeito: Construtor para campos final ou `@NonNull`. | Uso mostrado no material: `Service` com repositório final.

Anotação: `@Builder` | Código gerado ou efeito: API encadeada que nomeia cada valor definido. | Uso mostrado no material: Criação de `Pedido`.

Anotação: `@ToString` | Código gerado ou efeito: Gera representação textual do objeto. | Uso mostrado no material: Inclui apenas campos escolhidos.

Anotação: `@Data` | Código gerado ou efeito: Combina getters/setters, `toString`, `equals`/`hashCode` e construtor requerido. | Uso mostrado no material: O PDF explica o atalho e os cuidados; não é usado nas entidades do projeto.

Anotação: `@Slf4j` | Código gerado ou efeito: Gera um logger chamado `log`. | Uso mostrado no material: Conceito do PDF; não está aplicado nas classes desta fase.

Por que não usar @Data automaticamente em entidades

Em um relacionamento bidirecional, Cliente possui uma lista de Pedidos e cada Pedido aponta de volta para Cliente. Um toString() que percorra os dois lados pode entrar em recursão; equals e hashCode sobre todos os campos também podem percorrer relações ou acionar carregamento tardio. Por isso as entidades separam as anotações e controlam quais campos entram na representação textual.

O modelo implementado

Cliente usa @Getter, @Setter, @NoArgsConstructor e @ToString(onlyExplicitlyIncluded = true). Id, nome e idade são incluídos no texto; a coleção de pedidos não é. A lista é inicializada como ArrayList.

Pedido também gera getters/setters, construtores vazio e completo, Builder e toString limitado a id, descrição, valor e data. O campo Cliente é uma relação @ManyToOne(fetch = FetchType.LAZY); do outro lado, Cliente declara @OneToMany(mappedBy = "cliente"). Em JPA, o lado de Pedido com @ManyToOne é o lado dono da relação.

Leitura do Builder

```
Pedido pedido = Pedido.builder()
    .descricao("Curso de Java")
    .valor(new BigDecimal("199.90"))
    .dataCriacao(LocalDateTime.now())
    .cliente(cliente)
    .build();
```

O Builder deixa cada valor associado ao nome do campo e evita depender da ordem de muitos argumentos posicionais. BigDecimal representa valores decimais monetários com mais controle que double. LocalDateTime.now() captura a data e hora do momento em que o exemplo executa.

MainLombok cria Ana, constrói o pedido, adiciona o pedido à lista de Ana e imprime ambos. Há uma diferença importante no exemplo atual: ele não executa pedido.setCliente(cliente). Portanto, a lista do Cliente contém o Pedido, mas a referência inversa do Pedido ainda fica nula. Para manter o objeto coerente em memória, atualizam-se os dois lados:

```
pedido.setCliente(cliente);
cliente.getPedidos().add(pedido);
```

Além disso, o programa de Lombok é uma demonstração executada pelo IntelliJ, não a aplicação Spring principal do JAR. O Service usa @RequiredArgsConstructor para o repositório final, mas nesta fase não há um Controller próprio nem uma classe principal Spring no pacote Lombok.

ATENCAO

Diferença entre proposta e código: o PDF recomenda considerar igualdade de entidades pelo id, por exemplo com @EqualsAndHashCode(of = "id"), e propõe refatorar Service e Controller. O código atual evita @Data e limita o toString(), mas não implementa equals/hashCode por id e não possui Controller Lombok.

Como explicar esta fase

FALA SUGERIDA

"Lombok gera boilerplate na compilação. Usei getters, setters e construtores nas entidades; RequiredArgsConstructor mantém a injeção por construtor com dependência final; e Builder nomeia os campos ao criar um Pedido. Evitei Data nas entidades e restringi ToString para não percorrer o relacionamento Cliente-Pedido."

Mapa do projeto e demonstração

Esta seção ajuda a abrir o arquivo certo e evita confundir os exemplos de cada fase.

Arquivos que representam cada etapa

Fase | Arquivos principais | Configuração IntelliJ

Fase: 0 | Arquivos principais: Fase0/Main.java, Animal.java, Cachorro.java, Gato.java | Configuração IntelliJ: "Main" -> Fase0.Main

Fase: 1 | Arquivos principais: Fase1/Main.java e subpacotes Heranca, Abstracao, Interface, Sobrecarga e Sobrescricao | Configuração IntelliJ: "Fase1" -> Fase1.Main; há configurações adicionais por tópico

Fase: 2 | Arquivos principais: Fase2/Maven/Main.java, pom.xml | Configuração IntelliJ: "Fase2 - Maven" -> Fase2.Maven.Main

Fase: 3 | Arquivos principais: Fase3/MVC/MainMVC.java e classes Cliente* | Configuração IntelliJ: "Fase3 - MVC" -> Fase3.MVC.MainMVC

Fase: 4 | Arquivos principais: AplicacaoSpringBoot.java, Controller, Service, Repository, Model e application.properties | Configuração IntelliJ: "Fase4 - Spring Boot"

Fase: 5 | Arquivos principais: Fase5/Lombok/MainLombok.java, Model, Service e Repository | Configuração IntelliJ: "Fase5 - Lombok" -> Fase5.Lombok.MainLombok

Roteiro de demonstração em seis passos

1. Fase 0: execute os sons dos animais e explique tipo declarado versus tipo real.
2. Fase 1: mostre Fase1.Main; apresente abstrata/interface e compare overload com override.
3. Fase 2: abra o POM e execute a classe que usa StringUtils.isBlank.
4. Fase 3: percorra Controller -> Service -> Repository; cadastre um maior e tente um menor.
5. Fase 4: suba a aplicação, envie POST com JSON e consulte GET em localhost:8080/clientes.
6. Fase 5: execute MainLombok e explique o Builder e as anotações geradas.

Para a demonstração HTTP, inicie somente a configuração da fase 4 e aguarde o servidor ficar pronto antes de chamar a rota. O banco H2 é reiniciado junto com o processo.

Roteiro oral de aproximadamente quatro minutos

FALA SUGERIDA

"Comecei configurando Java 21 e Maven no IntelliJ e verificando que eu conseguia executar um main. Em seguida pratiquei orientação a objetos: herança reutiliza comportamento, abstração define o que as subclasses precisam implementar, interfaces definem capacidades, overload escolhe pela lista de parâmetros e override redefine uma implementação. Depois usei Maven para declarar e resolver bibliotecas. Na fase MVC separei dados, entrada, regra e armazenamento, com validação no Service e um Repository em memória. Na fase seguinte mantive essas responsabilidades e usei Spring Boot para criar os beans, expor POST e GET, converter JSON e persistir Cliente com JPA no H2. Por fim usei Lombok para gerar métodos repetitivos e Builder para construir Pedido de forma legível. Cada etapa acrescenta uma ferramenta ou uma forma de organização sem apagar o fundamento aprendido antes."

Perguntas que podem aparecer

Pergunta | Resposta curta e correta

Pergunta: Por que uma variável Animal pode emitir "Latido"? | Resposta curta e correta: Porque ela referencia um objeto Cachorro; o método sobrescrito é escolhido pelo tipo real durante a execução.

Pergunta: Qual a diferença entre classe abstrata e interface? | Resposta curta e correta: A classe abstrata pode compartilhar estado e código-base, além de exigir métodos. A interface define capacidades/contratos e uma classe pode implementar várias.

Pergunta: Como diferenciar overload de override? | Resposta curta e correta: Overload mantém o nome e muda parâmetros. Override preserva assinatura herdada e troca a implementação na subclasse.

Pergunta: Por que validar no Service? | Resposta curta e correta: Porque a regra deve valer para qualquer ponto de entrada; se ficasse no Controller, outro fluxo precisaria duplicá-la.

Pergunta: O que muda da fase 3 para a 4? | Resposta curta e correta: Na fase 3 a montagem é manual e o Repository usa mapa. Na fase 4 o Spring injeta dependências, o Controller atende HTTP e o Spring Data JPA usa H2.

Pergunta: O dado do H2 permanece após reiniciar? | Resposta curta e correta: Não. A URL usa H2 em memória e o esquema está configurado como create-drop.

Pergunta: O projeto tem todos os endpoints CRUD? | Resposta curta e correta: Não. A API implementa POST para cadastrar e GET para listar. Atualização e remoção aparecem como conceitos no PDF, não no código atual.

Pergunta: O que @RequestBody faz? | Resposta curta e correta: Recebe o corpo da requisição e converte o JSON para um objeto Java antes de chamar o método do Controller.

Pergunta: Lombok cria métodos durante a execução? | Resposta curta e correta: Não. O processador de anotações gera código durante a compilação, que fica no bytecode resultante.

Pergunta: Por que evitar @Data em entidades relacionadas? | Resposta curta e correta: Porque equals/hashCode e toString sobre todos os campos podem percorrer relações bidirecionais, causando recursão ou carregamento inesperado.

Pergunta: O relacionamento de MainLombok está completo? | Resposta curta e correta: A lista `Cliente.pedidos` é atualizada, mas `pedido.cliente` não. Para coerência em memória, deve-se atribuir a referência nos dois lados.

Pergunta: Qual aplicação o JAR inicia? | Resposta curta e correta: O POM configura `Fase4.SpringBoot.AplicacaoSpringBoot` como classe principal do plugin Spring Boot.

Estado atual e melhorias possíveis

As fases demonstrativas usam métodos `main` e chamadas manuais; não foi encontrada uma suíte automatizada em `src/test`. A fase 3 armazena dados somente em memória. A fase 4 ainda não tem GET por id, atualização, remoção, validação declarativa nem handler de erros. Na fase 5, a associação do exemplo deve ser sincronizada nos dois lados se for usada para persistência. Esses pontos são próximos passos; não alteram o que cada fase já demonstra.

Glossário rápido

Termo | Significado

Termo: Bean | Significado: Objeto criado e gerenciado pelo Spring.

Termo: Injeção de dependência | Significado: Entrega de uma dependência necessária a uma classe, por exemplo pelo construtor.

Termo: JAR | Significado: Pacote Java que reúne classes e recursos para distribuição/execução.

Termo: JPA | Significado: Especificação Java para mapear objetos a tabelas relacionais.

Termo: Polimorfismo | Significado: Uso de um tipo comum com comportamentos diferentes conforme o objeto concreto.

Termo: Boilerplate | Significado: Código repetitivo de estrutura, como getters e construtores.

Termo: DTO | Significado: Objeto usado para transportar dados entre partes de um sistema; o projeto atual recebe a própria entidade no POST.

Materiais analisados

- `fases_PDF/fase0-intellij 1.pdf` até `fases_PDF/fase5-lombok.pdf` - 28 páginas ao todo.
- Código em `src/main/java/Fase0` a `Fase5`, respectivos READMEs, `pom.xml` e `src/main/resources/application.properties`.
- Configurações de execução do IntelliJ em `.idea/runConfigurations`.

PONTO-CHAVE

Para encerrar a apresentação: retome a progressão da trilha - primeiro executar, depois modelar, organizar dependências, separar responsabilidades, expor uma API e reduzir código repetitivo.